Assignment: Online A/B Experiment

Randomization Check

Here is the first few rows of our data-set.

```r
library(readr)
df <- read_csv("data_new_algo.csv")
```

```
Rows: 10000 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (7): user_id, log_in_day, prior_minutes, new_algo, minutes, churn_prob, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(df, 7)
```

```
# A tibble: 7 x 7
  user_id log_in_day prior_minutes new_algo minutes churn_prob churn
    <dbl>      <dbl>         <dbl>    <dbl>   <dbl>      <dbl> <dbl>
1       1          3         424.         1   446.        0.08     1
2       2          3         253.         1   220.        0.08     0
3       3          3         183.         1   182.        0.08     0
4       4          2         472.         1   583.        0.08     0
5       5          3          60.7        0    49.3       0.1      0
6       6          2         463.         1   440.        0.08     0
7       7          2         602.         0   594.        0.1      0
```

Based on our problem, we can assign:

H0(Null-Hypothesis): There is no significant effect of new algorithm.

H1(Alternate-Hypothesis): There is a significant effect of new algorithm.

Balance check for overall prior minutes and for individual day:

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
# 1. Overall summary (all users)
overall_summary <- df %>%
  summarise(
    n = n(),
    mean_prior = mean(prior_minutes, na.rm = TRUE),
    sd_prior = sd(prior_minutes, na.rm = TRUE),
    .groups = "drop"
  )

# 2. Summary by treatment group
treatment_summary <- df %>%
  group_by(new_algo) %>%
  summarise(
    n = n(),
    mean_prior = mean(prior_minutes, na.rm = TRUE),
    sd_prior = sd(prior_minutes, na.rm = TRUE),
    .groups = "drop"
  )

# 3. Summary by treatment group and log_in_day
daily_treatment_summary <- df %>%
```

```r
  group_by(log_in_day, new_algo) %>%
  summarise(
    n = n(),
    mean_prior = mean(prior_minutes, na.rm = TRUE),
    sd_prior = sd(prior_minutes, na.rm = TRUE),
    .groups = "drop"
  )


# overall_summary
```

```r
treatment_summary
```

```
# A tibble: 2 x 4
  new_algo     n mean_prior sd_prior
     <dbl> <int>      <dbl>    <dbl>
1        0  5037       399.     216.
2        1  4963       400.     216.
```

In terms of overall summary, we can see the difference is 0.75 minutes which is quite balanced.

```r
daily_treatment_summary
```

```
# A tibble: 6 x 5
  log_in_day new_algo     n mean_prior sd_prior
       <dbl>    <dbl> <int>      <dbl>    <dbl>
1          1        0  1661       491.     204.
2          1        1  1674       505.     203.
3          2        0  1677       404.     200.
4          2        1  1637       395.     194.
5          3        0  1699       304.     201.
6          3        1  1652       297.     198.
```

On each day the difference went lower and the values are 14.53, 8.28, 7.58 respectively. So the difference declined gradually. However, this notable difference tells us that on daily basis it was a little imbalanced at least in initial days.

T-test:

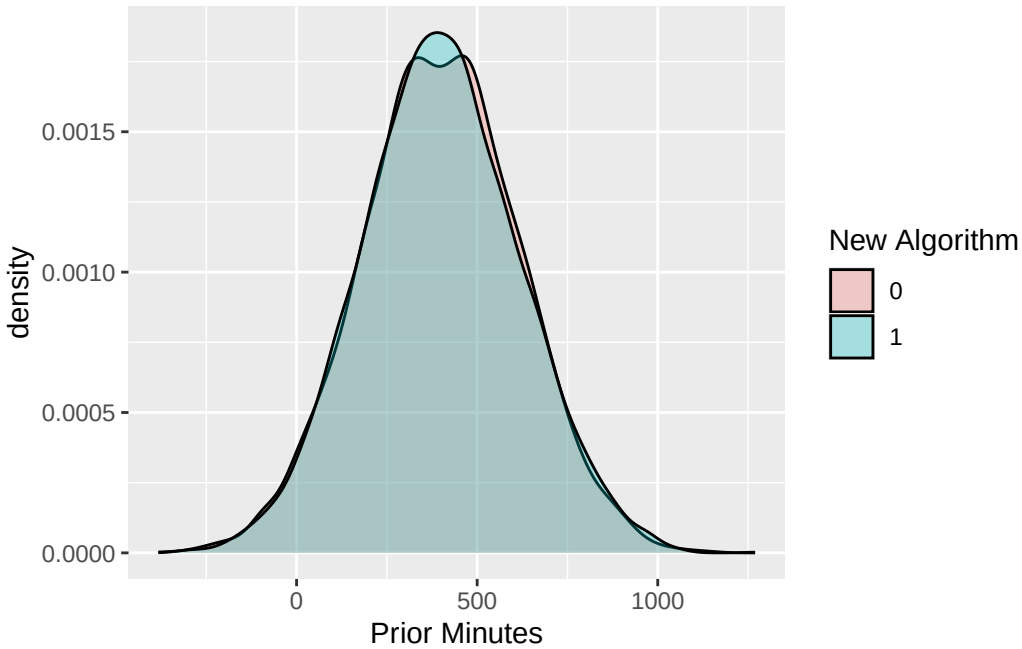
```r
library(broom)

# Run t-test: compare prior_minutes between new_algo groups
t_test_prior_overall <- t.test(prior_minutes ~ new_algo, data = df)

# Clean the output using broom::tidy
t_test_result <- tidy(t_test_prior_overall)

# View result
t_test_result
```

```
# A tibble: 1 x 10
  estimate estimate1 estimate2 statistic p.value parameter conf.low conf.high
     <dbl>     <dbl>     <dbl>     <dbl>   <dbl>     <dbl>    <dbl>     <dbl>
1   -0.742      399.      400.    -0.172   0.863     9995.    -9.20      7.72
# i 2 more variables: method <chr>, alternative <chr>
```

Our P-value is 86% which is quite higher than our threshold alpha=0.05. Hence based on this result we can not say, we have significant change in mean minutes.

Density Plot

```r
# Density plot
library(ggplot2)
ggplot(df, aes(x=prior_minutes, fill=factor(new_algo))) +
  geom_density(alpha=0.3) +
  labs(fill="New Algorithm", x="Prior Minutes")
```



Since, our density plot also overlaps with each other, we can conclude that the randomization done here is working or the quality of the randomization is approved here.

Power Analysis

```
library(readr)
df <- read_csv("data_new_algo.csv")
```

```
Rows: 10000 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (7): user_id, log_in_day, prior_minutes, new_algo, minutes, churn_prob, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
nrow(df)
```

```
[1] 10000
```

Two-sample T-Test: (Minutes)

```
library(pwr)
alpha        <- 0.05
power_target <- 0.80
n_per_arm    <- nrow(df)/2   #   5,000 users per arm
# Observed sd
sd_minutes <- sd(df$minutes)


# 1) MDE: solve for d then × sd
d_mde <- pwr.t.test(
  n         = n_per_arm,
  sig.level = alpha,
  power     = power_target,
  type      = "two.sample"
)$d


mde <- d_mde * sd_minutes


# 2) Required n to detect Δ = 10
d_mde_target <- 10 / sd_minutes
n_req <- pwr.t.test(
  d         = d_mde_target,
  sig.level = alpha,
  power     = power_target,
  type      = "two.sample"
)$n

# 2) Required n to detect Δ = 0.03
d_mde_target_small <- 0.03 / sd_minutes
n_req_small<- pwr.t.test(
  d         = d_mde_target_small,
  sig.level = alpha,
  power     = power_target,
  type      = "two.sample"
)$n
# Print results
# cat(sprintf("T-Test: %.3f\n", d_mde))
cat(sprintf("MDE = %.3f\n", mde), sprintf("When required n per arm for (MDE=10): %.0f\n", n_r
```

```
MDE = 12.534
 When required n per arm for (MDE=10): 7854
```

```
cat(sprintf("MDE = %.3f\n", mde), sprintf("When required n per arm for (MDE=0.03): %.0f\n", n
```

```
MDE = 12.534
 When required n per arm for (MDE=0.03): 872525076
```

Based on this results,

1. In this current setting when Alpha=0.05 and Power=80, Sample Size per arm=5000 our MDE is 12.534minutes. Therefore we need at least 12.534 minutes difference between two groups(control & treatment) in order to detect the effect reliably.

2. We need 7854 users per arm(in control and in treatment group) to reliably detect a change when we set MDE =10 minutes.

   However when the effect size is too small (e.g. 0.01) its giving an error. As the output is becoming too big. However there is a limit, we can not make our effect size smaller than 0.03. Even for this small effect we need 872525076 users which is a lot.

   For **more smaller effect change** in minutes we need **more users or samples**.

Two proportion z-test: (Churn):

That's a simple code chunk where we want to find the churn rate which is necessary for the purpose of measuring two proportion z test.

```
churn_rate_by_algo <- df %>%
  group_by(new_algo) %>% # Group by the 'new_algo' variable
  summarise(
    churned_users = sum(churn == 1, na.rm = TRUE), # Count churned users in each group
    total_users   = n(),                           # Count total users in each group
    churn_rate    = (churned_users / total_users) * 100 # Calculate churn rate for each grou
  ) %>%
  ungroup()

cat("Churn Rate by new_algo Group:\n")
```

```
Churn Rate by new_algo Group:
```

```r
print(churn_rate_by_algo)
```

```
# A tibble: 2 x 4
  new_algo churned_users total_users churn_rate
     <dbl>         <int>       <int>      <dbl>
1        0           515        5037      10.2
2        1           384        4963       7.74
```

```r
cat("\n")
```

Here we can see churn rate is more than the users who are still continuing. We need control group churn rate or baseline churn which is 0.10.

```r
library(pwr)

# Inputs
baseline_churn <- 0.102
n_per_group <- 5000
alpha <- 0.05
power_target <- 0.80

# Solve for effect size (Cohen's h)
mde_result <- pwr.2p.test(
  h = NULL,
  n = n_per_group,
  sig.level = alpha,
  power = power_target,
  alternative = "two.sided"
)
mde_h <- mde_result$h

# Function to invert Cohen's h
invert_ESh <- function(p1, h_value) {
  uniroot(function(p2_val) {
    if (p2_val < 0 || p2_val > 1) return(NA)
    ES.h(p1, p2_val) - h_value
  }, interval = c(0.000001, 0.999999))$root
}

# Compute absolute churn rate changes
mde_diff_increase <- abs(invert_ESh(baseline_churn, mde_h) - baseline_churn)
```

```r
mde_diff_decrease <- abs(baseline_churn - invert_ESh(baseline_churn, -mde_h))

# Take the larger absolute difference
mde_churn_difference <- max(mde_diff_increase, mde_diff_decrease)

# Print the result in percentage
cat(sprintf("The Minimum Detectable Churn Rate difference: %.3f%%\n", mde_churn_difference *
```

```
The Minimum Detectable Churn Rate difference: 1.758%
```

Our finding here 1.758% which is MDE(churn rate) tells us minimum difference in churn rates between the two groups that our experiment can reliably detect, given the input that we set(alpha, power, N).

```r
# Parameters
p      <- 0.1         # baseline proportion (e.g., churn rate)
delta_original  <- 0.0176
delta_ten  <- 10
delta_small  <- 0.01
power  <- 0.80        # desired power
z_alpha <- 1.96
z_beta <- 0.84

# Sample size per group
n <- (2 * p * (1 - p) * (z_alpha + z_beta)^2) / (delta_original^2)
n10 <- (2 * p * (1 - p) * (z_alpha + z_beta)^2) / (delta_ten^2)
nsmall <- (2 * p * (1 - p) * (z_alpha + z_beta)^2) / (delta_small^2)

# Total sample size (2 groups)
# n_total <- ceiling(2 * n)

cat(sprintf("  Required sample when Delta is original(0.0174): %.0f\n", n ))
```

```
  Required sample when Delta is original(0.0174): 4556
```

```r
cat(sprintf("  Required sample when Delta=0.01: %.0f\n", nsmall ))
```

```
  Required sample when Delta=0.01: 14112
```

```r
cat(sprintf("  Required sample when Delta=10: %.4f\n", n10 ))
```

```
  Required sample when Delta=10: 0.0141
```

We can simply observe that, we need **more samples/users** in order to compute for a **minimal change** in effect. On the other hand, unlike previous calculations this one gave almost 0 for bigger effect.

Group Mean Comparison

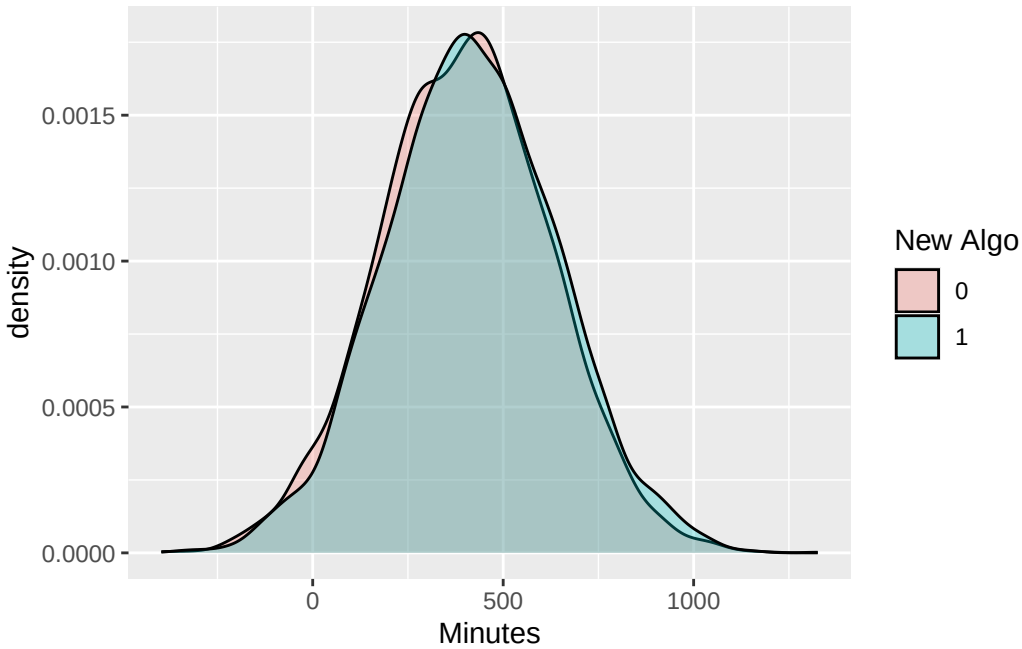
For minutes:

```r
library(dplyr)

# Inspect treatment groups
df %>%
  group_by(new_algo) %>%
  summarise(
    n = n(),
    mean = mean(minutes),
    sd = sd(minutes)
  ) %>%
  ungroup()
```

```
# A tibble: 2 x 4
  new_algo     n  mean    sd
     <dbl> <int> <dbl> <dbl>
1        0  5037  399.  222.
2        1  4963  418.  225.
```

We can see that there is a notable difference between the two mean where as the Standard deviation is pretty closer. That 19.08 minutes mean difference tends us to reject the null hypothesis although we must need to delve into further analysis and interpretations.

```r
# Density plot
library(ggplot2)
ggplot(df, aes(x=minutes, fill=factor(new_algo))) +
  geom_density(alpha=0.3) +
  labs(fill="New Algo", x="Minutes")
```



This visualization does not depicts the closeness of two groups as we can see few gaps between them at many points. Here we have concrete proof to reject null hypothesis.

```
library(broom)
# t-test
t_test <- t.test(minutes ~ new_algo, data = df)
tidy(t_test)
```

```
# A tibble: 1 x 10
  estimate estimate1 estimate2 statistic   p.value parameter conf.low conf.high
     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1    -19.1      399.      418.     -4.27 0.0000198     9989.    -27.8     -10.3
# i 2 more variables: method <chr>, alternative <chr>
```

```
t_test_99 <- t.test(minutes ~ new_algo, data = df, conf.level = 0.99)
tidy(t_test_99)
```

```
# A tibble: 1 x 10
  estimate estimate1 estimate2 statistic   p.value parameter conf.low conf.high
     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1    -19.1      399.      418.     -4.27 0.0000198     9989.    -30.6     -7.57
# i 2 more variables: method <chr>, alternative <chr>
```

Here we computed the T test with a default 95% and an additional CI of 99%. At 99% interval we are getting a bigger interval than the 95%. However, P-value remained same for both of them which is (<0.05).

As a result, we can say that this result is **statistically significant.**

For churn:

```
library(dplyr)
library(broom)
# Summarize churn
churn_summary <- df %>%
  group_by(new_algo) %>%
  summarise(
    churned = sum(churn == 1, na.rm = TRUE),
    total   = n()
  )

# Print the summary
print(churn_summary)
```

```
# A tibble: 2 x 3
  new_algo churned total
     <dbl>   <int> <int>
1        0     515  5037
2        1     384  4963
```

```
# Extract values (using 0 for control, 1 for treatment)
x1 <- churn_summary$churned[churn_summary$new_algo == 0]  # control
x2 <- churn_summary$churned[churn_summary$new_algo == 1]  # treatment
n1 <- churn_summary$total[churn_summary$new_algo == 0]
n2 <- churn_summary$total[churn_summary$new_algo == 1]

# # Check values
# print(c(x1, x2))
# print(c(n1, n2))

# Run proportion test
prop_test_95 <- prop.test(x = c(x1, x2), n = c(n1, n2), conf.level = 0.95)
tidy(prop_test_95)
```

```
# A tibble: 1 x 9
  estimate1 estimate2 statistic   p.value parameter conf.low conf.high method
      <dbl>     <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl> <chr>
1     0.102    0.0774      18.6 0.0000162         1   0.0135    0.0363 2-sample~
# i 1 more variable: alternative <chr>
```

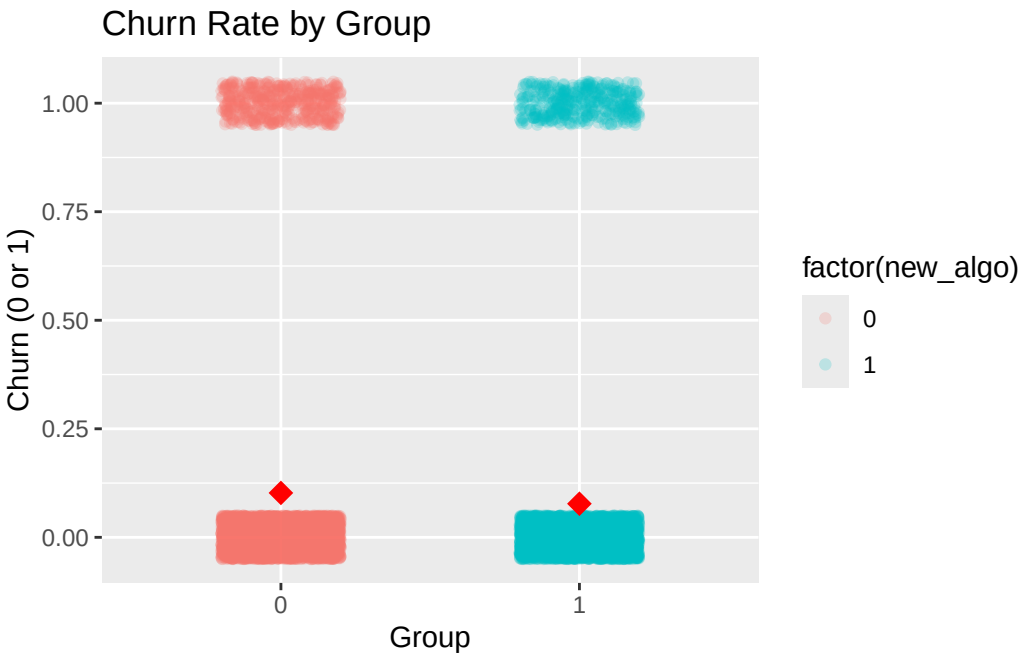
```r
# Optional: 99% confidence
prop_test_99 <- prop.test(x = c(x1, x2), n = c(n1, n2), conf.level = 0.99)
tidy(prop_test_99)
```

```
# A tibble: 1 x 9
  estimate1 estimate2 statistic   p.value parameter conf.low conf.high method
      <dbl>     <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl> <chr>
1     0.102    0.0774      18.6 0.0000162         1  0.00996    0.0398 2-sample~
# i 1 more variable: alternative <chr>
```

The difference in churn rates between the control (10.22%) and treatment (7.74%) groups is statistically significant (**$p < 0.001$**) with an estimated reduction of about **2.85 percentage points**. If we run the experiment 100 times we will get the true effect size between(1.35-3.63)%.

However for 99% interval the interval is bigger and its unusual as the level of uncertainty will increase.

```r
library(ggplot2)
ggplot(df, aes(x = factor(new_algo), y = churn, color = factor(new_algo))) +
  geom_jitter(width = 0.2, height = 0.05, alpha = 0.2) +
  stat_summary(fun = mean, geom = "point", shape = 18, size = 4, color = "red") +
  labs(title = "Churn Rate by Group", x = "Group", y = "Churn (0 or 1)")
```



We can not visualize binary variable with density plotting hence we have chosen **Jitter plot** for better understanding. Here we can see the red diamond dot is close to 0 in terms of treatment group. Hence we have good evidence to reject null hypothesis as we have P<0.01.

Hypothesis based on 5% more listening time:

```
library(broom)
# 1. Compute mean of control group
mean_control <- mean(df$minutes[df$new_algo == 0])

# 2. Compute threshold = 5% increase over control
threshold <- mean_control * 1.05

# 3. One-sided t-test: is treatment mean > 5% above control?
result01 <- t.test(
  df$minutes[df$new_algo == 1],     # treatment group
  mu = threshold,                   # minimum acceptable mean
  alternative = "greater"           # one-sided test
)
tidy(result01)
```

```
# A tibble: 1 x 8
  estimate statistic p.value parameter conf.low conf.high method     alternative
```

	(1)
(Intercept)	est = 399.374 (se = 3.125, t = 127.804)***
	p = <0.001, CI = [393.249, 405.500]
new_algo	est = 19.085 (se = 4.470, t = 4.269)***
	p = <0.001, CI = [10.322, 27.848]
R2	0.002

```
     <dbl>     <dbl>   <dbl>     <dbl>    <dbl>     <dbl> <chr>      <chr>
1     418.    -0.276   0.609      4962     413.       Inf One Sampl~ greater
```

Regression Analysis

For minutes

```
# OLS Regression
library(modelsummary)
reg <- lm(minutes ~ new_algo, data = df)
modelsummary(reg, vcov = sandwich::vcovHC,
             estimate = "est = {estimate} (se = {std.error}, t = {statistic}){stars}",
             statistic = "p = {p.value}, CI = [{conf.low}, {conf.high}]",
             gof_map = c("r.squared"))
```

We can interpret this result as, when New Algorithm(no uses) is 0, people listens 399.37minutes on average. The new algorithm shows a highly significant effect (p<0.001). Since our model predicts, users spend 19.09 more minutes.

In terms of Confidence Interval, If we run the same experiment many time 95% time the extra minutes causes by new algorithm will be found between the intervals(10.32-27.85).

On the contrary, R-square tell us that the new algorithm explains only 0.2% of the overall variation in how many users listen and other cases, it might be happening because of other factors(competitors, music content, leisure time, season).

For churn

	(1)
(Intercept)	log(OR) = −2.173 (se = 0.047)***
	p = <0.001, CI = [−2.264, −2.081]
new_algo	log(OR) = −0.306 (se = 0.071)***
	p = <0.001, CI = [−0.444, −0.168]
Num.Obs.	10 000
AIC	6031.2
BIC	6045.6
Log.Lik.	−3013.581
RMSE	0.29
Std.Errors	Custom

```r
# Load required libraries
library(margins)
library(modelsummary)
library(sandwich)

# Logistic regression model
logit_model <- glm(churn ~ new_algo, data = df, family = binomial)

# Model summary with robust standard errors
modelsummary(
  logit_model,
  vcov = sandwich::vcovHC,
  estimate = "log(OR) = {estimate} (se = {std.error}){stars}",
  statistic = "p = {p.value}, CI = [{conf.low}, {conf.high}]"
)
```

```r
margins_result <- margins(logit_model)
summary(margins_result)
```

```
   factor     AME     SE       z      p   lower   upper
 new_algo -0.0250 0.0058 -4.3175 0.0000 -0.0363 -0.0136
```

Since churn is a binary variable we used logistic regression here.

Here in Intercept(baseline) it means

Odds Ratio = Odds of outcome in group 1 / Odds of outcome in group 2 @gfg_odds_ratio

e(-2.173)=0.114 means users not likely(less than 1) to churn.

Meanwhile when users are exposed to new algorithm, we can measure

Odds of churning=baseline odds**odds ratio of new algo. @ucla_odds_ratio*

*=(0.114**0.736)

= 0.0838 Which we can say very low value(churn likelihood) in terms of those user who exposed new algorithm as P=<0.01.

AME:

```
# 1. Load required libraries
library(margins)   # For AME
library(dplyr)

# 2. Fit logistic regression model (adjust outcome/covariates as needed)
model_logit <- glm(churn ~ new_algo + log_in_day + prior_minutes, data = df, family = binomia

# 3. Compute Average Marginal Effects (AME)
ame_result <- margins(model_logit)

# 4. View AME summary
summary(ame_result)
```

```
        factor     AME     SE       z      p   lower   upper
    log_in_day  0.0011 0.0038  0.2797 0.7797 -0.0063  0.0084
      new_algo -0.0250 0.0058 -4.3140 0.0000 -0.0363 -0.0136
 prior_minutes -0.0000 0.0000 -1.0478 0.2947 -0.0000  0.0000
```

```
# Optional: extract AME for new_algo specifically
summary(ame_result) %>% filter(factor == "new_algo")
```

```
   factor     AME     SE       z      p   lower   upper
 new_algo -0.0250 0.0058 -4.3140 0.0000 -0.0363 -0.0136
```

According to the AME=2.5%, we can say the new algorithm reduce churn by 2.5%. This estimate is based on the **Average Marginal Effect (AME)**, which reflects the average change in predicted probability when moving from the control group to the treatment group.

	(1)
(Intercept)	est = −4.327 (se = 2.050, t = −2.110)*
	p = 0.035, CI = [−8.345, −0.308]
new_algo	est = 18.340 (se = 0.997, t = 18.401)***
	p = <0.001, CI = [16.387, 20.294]
log_in_day	est = 0.409 (se = 0.663, t = 0.617)
	p = 0.538, CI = [−0.891, 1.709]
prior_minutes	est = 1.010 (se = 0.002, t = 404.980)***
	p = <0.001, CI = [1.005, 1.015]
R2	0.950

Variance reduction

Variance reduction for minutes

```
# OLS Regression
library(modelsummary)
reg <- lm(minutes ~ new_algo + log_in_day +prior_minutes, data = df)
modelsummary(reg, vcov = sandwich::vcovHC,
             estimate = "est = {estimate} (se = {std.error}, t = {statistic}){stars}",
             statistic = "p = {p.value}, CI = [{conf.low}, {conf.high}]",
             gof_map = c("r.squared"))
```

After adding covariates, "log_in_day" and "prior_minutes", we see that the **standard error (SE)** for new_algo **decreased**, leading to a **narrower confidence interval**, and the model's R-square jumped to 95%.

We can say now, 95% times users listening because of new algorithms which makes our experiment more reliable.

Variance reduction for Churn

```
library(margins)
library(modelsummary)
library(sandwich)

# Logistic regression model
logit_model <- glm(churn ~ new_algo + log_in_day +prior_minutes, data = df, family = binomial
```

	(1)
(Intercept)	log(OR) = −2.126 (se = 0.144)***
	p = <0.001, CI = [−2.408, −1.844]
new_algo	log(OR) = −0.306 (se = 0.071)***
	p = <0.001, CI = [−0.444, −0.167]
log_in_day	log(OR) = 0.013 (se = 0.047)
	p = 0.783, CI = [−0.079, 0.105]
prior_minutes	log(OR) = 0.000 (se = 0.000)
	p = 0.291, CI = [−0.001, 0.000]
Num.Obs.	10 000
AIC	6033.5
BIC	6062.4
Log.Lik.	−3012.772
RMSE	0.29
Std.Errors	Custom

```r
# Model summary with robust standard errors
modelsummary(
  logit_model,
  vcov = sandwich::vcovHC,
  estimate = "log(OR) = {estimate} (se = {std.error}){stars}",
  statistic = "p = {p.value}, CI = [{conf.low}, {conf.high}]"
)
```

After adding covariates we can see a change in standard error and the confidence interval has also increased.

CUPED

```r
library(dplyr)       # or
library(tidyverse)   # which includes dplyr and more
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.0     v stringr   1.5.1
```

	(1)
(Intercept)	est = 399.746 (se = 0.695, t = 575.582)***
	p = <0.001, CI = [398.385, 401.108]
new_algo	est = 18.336 (se = 0.997, t = 18.399)***
	p = <0.001, CI = [16.382, 20.289]
R2	0.033

```
v lubridate 1.9.4     v tibble    3.2.1
v purrr     1.0.4     v tidyr     1.3.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
# 1) Compute theta_hat
theta_hat <- lm(minutes~prior_minutes, data = df)$coefficients[2]

# 2) Adjust outcomes
df <- df %>% mutate(minute_cuped = minutes - theta_hat * (prior_minutes - mean(prior_minutes

# 3) OLS Regression
library(modelsummary)
reg <- lm(minute_cuped ~ new_algo, data = df)
modelsummary(reg, vcov = sandwich::vcovHC,
             estimate = "est = {estimate} (se = {std.error}, t = {statistic}){stars}",
             statistic = "p = {p.value}, CI = [{conf.low}, {conf.high}]",
             gof_map = c("r.squared"))
```

After CUPED computation we can compare it to the regular variance reduction and end up by saying that CUPED also remained Standard error and Confidence Interval smaller.

But unlike previous method, R-square has decreased to 3.3%. Based on that new algorithm's significance also decreased.

Conditional Average Treatment Effect(CATE)

CATE Log In Day (Minutes-Linear regression):

Table 1: CATE by Log In Day

	Day_01	Day_02	Day_03
(Intercept)	491.56***	402.70***	305.68***
	(5.20)	(5.02)	(5.02)
new_algo	34.23***	10.08	8.94
	(7.34)	(7.13)	(7.16)
Num.Obs.	3334	3333	3333
R2	0.006	0.001	0.000

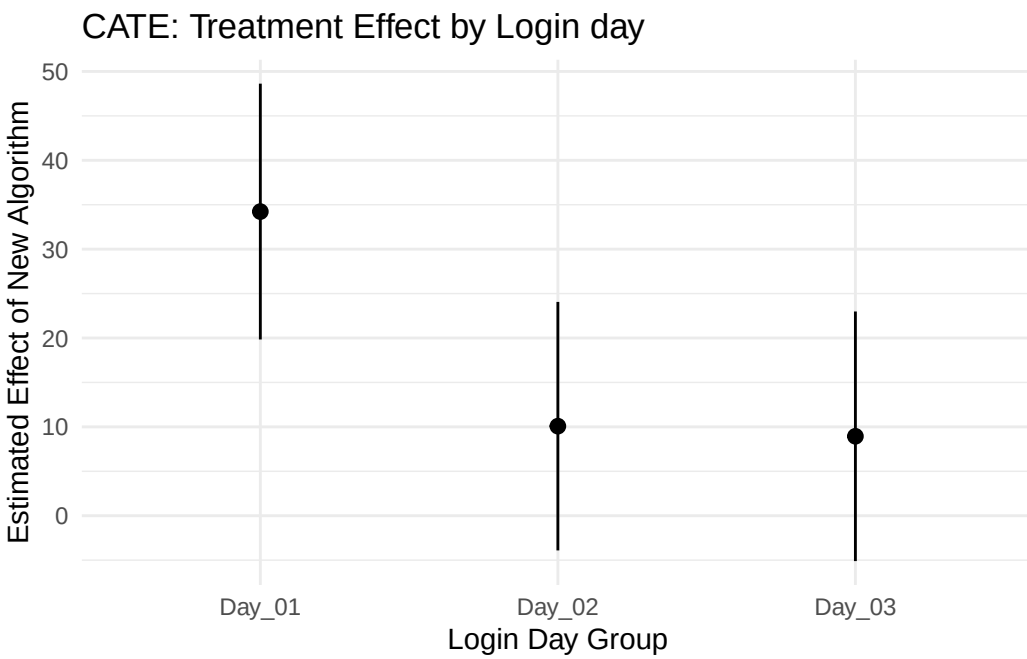
+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
# 0) Create day buckets
df_q <- df %>%
  mutate(loginday_q = factor(
      ntile(log_in_day, 3),
      levels = 1:3,
      labels = c("Day_01", "Day_02", "Day_03")
    ))

# 1) Regression analysis with subgroup comparison
model_group <- df_q %>%
  group_by(loginday_q) %>%
  group_map(~ lm(minutes ~ new_algo, data = .x)) %>%
  setNames(levels(df_q$loginday_q))

modelsummary(
  model_group,
  stars = TRUE,
  fmt = 2,
  title = "CATE by Log In Day",
  gof_map = c("nobs", "r.squared")
)
```

Here we have divided login day into 3 different groups that is day01, day 02 and day03. We can see a good decline in terms of baseline estimate. For day01 we see a significant effect size but other days it declined. A good business interpretation would be, people were more interested into the new algorithm as R-square 0.6%(99.4% case people listened more due to other reasons). Its not that significant but still better than other 2 days.

```r
library(dplyr)
library(broom)
library(ggplot2)
library(purrr)

# 0) Create login day buckets
df_q <- df %>%
  mutate(loginday_q = factor(
    ntile(log_in_day, 3),
    levels = 1:3,
    labels = c("Day_01", "Day_02", "Day_03")
  ))

# 1) Regression analysis with subgroup comparison
model_group <- df_q %>%
  group_by(loginday_q) %>%
  group_map(~ lm(minutes ~ new_algo, data = .x)) %>%
  setNames(levels(df_q$loginday_q))


coef_df <- map_dfr(
  model_group,
  ~ tidy(.x, conf.int = TRUE) %>%
    filter(term == "new_algo"),
  .id = "loginday_q"
)

expected_levels <- tibble(loginday_q = c("Day_01", "Day_02", "Day_03"))

coef_df_full <- expected_levels %>%
  left_join(coef_df, by = "loginday_q")

# 4) Plot with ggplot
ggplot(coef_df_full, aes(x = loginday_q, y = estimate)) +
  geom_pointrange(
    aes(ymin = conf.low, ymax = conf.high),
    fatten = 2, size = 0.8, na.rm = TRUE
  ) +
  scale_x_discrete(limits = c("Day_01", "Day_02", "Day_03")) +
  labs(
    x = "Login Day Group",
    y = "Estimated Effect of New Algorithm",
```

```
    title = "CATE: Treatment Effect by Login day"
  ) +
  theme_minimal()
```



This simplified graph highlights that the algorithm had a greater impact on Day 01 than on the following days.

CATE Log In Day (Minutes-Linear regression):

```
# 0) Create day buckets
df_q <- df %>%
  mutate(loginday_churn = factor(
    ntile(log_in_day, 3),
    levels = 1:3,
    labels = c("Day_01", "Day_02", "Day_03")
  ))

# 1) Logistic regression by log_in_day group
model_group_logit <- df_q %>%
  group_by(loginday_churn) %>%
  group_map(~ glm(churn ~ new_algo, data = .x, family = binomial(link = "logit"))) %>%
  setNames(levels(df_q$loginday_churn))
```

Table 2: CATE by Log In Day (Logistic Regression on Churn)

	Day_01	Day_02	Day_03
(Intercept)	−2.217***	−2.233***	−2.074***
	(0.082)	(0.082)	(0.077)
new_algo	−0.208+	−0.294*	−0.412***
	(0.122)	(0.125)	(0.121)
Num.Obs.	3334	3333	3333
AIC	2013.5	1945.1	2077.6

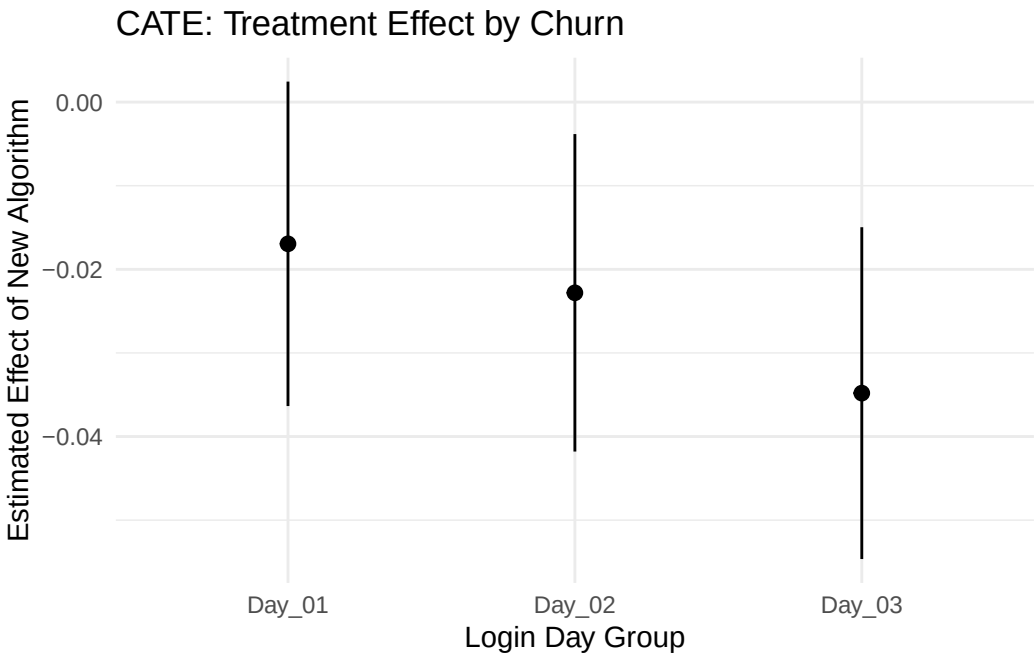
+ p <0.1, * p <0.05, ** p <0.01, *** p <0.001

```
# 2) Summarize models
modelsummary(
  model_group_logit,
  stars = TRUE,
  fmt = 3,
  title = "CATE by Log In Day (Logistic Regression on Churn)",
  gof_map = c("nobs", "aic")
)
```

We can see less churn on daily basis in terms of baseline estimate. The effect of new algorithm is noticeable in the basis of day. On day 03 the churn was lowest. Standard error remained same as before. P value is significantly lower than 0.05 which makes our experiment more significant.

```
library(dplyr)
library(broom)
library(ggplot2)
library(purrr)

# 0) Create login day buckets
df_q <- df %>%
  mutate(loginday_churn = factor(
    ntile(log_in_day, 3),
    levels = 1:3,
    labels = c("Day_01", "Day_02", "Day_03")
  ))
```

```r
# 1) Regression analysis with subgroup comparison
model_group <- df_q %>%
  group_by(loginday_churn) %>%
  group_map(~ lm(churn ~ new_algo, data = .x)) %>%
  setNames(levels(df_q$loginday_churn))


coef_df <- map_dfr(
  model_group,
  ~ tidy(.x, conf.int = TRUE) %>%
    filter(term == "new_algo"),
  .id = "loginday_churn"
)

expected_levels <- tibble(loginday_churn = c("Day_01", "Day_02", "Day_03"))

coef_df_full <- expected_levels %>%
  left_join(coef_df, by = "loginday_churn")

# 4) Plot with ggplot
ggplot(coef_df_full, aes(x = loginday_churn, y = estimate)) +
  geom_pointrange(
    aes(ymin = conf.low, ymax = conf.high),
    fatten = 2, size = 0.8, na.rm = TRUE
  ) +
  scale_x_discrete(limits = c("Day_01", "Day_02", "Day_03")) +
  labs(
    x = "Login Day Group",
    y = "Estimated Effect of New Algorithm",
    title = "CATE: Treatment Effect by Churn"
  ) +
  theme_minimal()
```



The results clearly show a gradual reduction in churn due to the new algorithm across Days 01 to 03.

CATE interaction

```
# 2) Regression analysis with interaction
model_int <- lm(prior_minutes ~ new_algo * minutes, data = df)

modelsummary(
  model_int,
  stars    = TRUE,
  fmt      = 2,
  title    = "CATE Interaction Model",
  estimate = "est = {estimate} (se = {std.error}, p = {p.value}){stars}",
  statistic = NULL,
  gof_map  = c("nobs", "r.squared")
)
```

We can see an effect size of 20.43minutes when our multiplicative variables are new_algo and log_in_day.

Having a good R-square that is 95%, makes our experiment more reliable.

Table 3: CATE Interaction Model

	(1)
(Intercept)	est = 20.43 (se = 1.40, p = <0.01)***
new_algo	est = −12.20 (se = 2.01, p = <0.01)***
minutes	est = 0.95 (se = 0.00, p = <0.01)***
new_algo × minutes	est = −0.01 (se = 0.00, p = <0.01)**
Num.Obs.	10 000
R2	0.950

```r
library(marginaleffects)

# 1) Average Marginal Effect
avg_slopes(model_int, variables = "new_algo")
```

```
 Estimate Std. Error     z Pr(>|z|)     S 2.5 % 97.5 %
    -17.2      0.963 -17.9   <0.001 235.4 -19.1  -15.3

Term: new_algo
Type: response
Comparison: 1 - 0
```

Bayesian A/B-testing

Minutes(Non-informative, Informative)

```r
library(rstanarm)
```

```
Loading required package: Rcpp

This is rstanarm version 2.32.1

- See https://mc-stan.org/rstanarm/articles/priors for changes to default priors!

- Default priors may change, so it's safest to specify priors, even if equivalent to the defa
```

```
- For execution on a local, multicore CPU with excess RAM we recommend calling

  options(mc.cores = parallel::detectCores())
```

```r
library(bayesplot)
```

```
This is bayesplot version 1.12.0

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to bayesplot::theme_default()

   * Does _not_ affect other ggplot2 plots

   * See ?bayesplot_theme_set for details on theme setting
```

```r
library(ggplot2)

# Clean theme and base font for PDF compatibility
theme_set(theme_minimal(base_family = "sans"))

# Load and prepare the data
df <- read.csv("data_new_algo.csv")
y_A <- df[df$new_algo == 0, "minutes"]
y_B <- df[df$new_algo == 1, "minutes"]

df_combined <- data.frame(
  minutes = c(y_A, y_B),
  new_algo = c(rep("A", length(y_A)), rep("B", length(y_B)))
)
df_combined$new_algo <- relevel(as.factor(df_combined$new_algo), ref = "A")

# Fit non-informative model
set.seed(123)
fit_noninf <- stan_glm(
  minutes ~ new_algo,
  data = df_combined,
  family = gaussian(),
  prior_intercept = normal(4, 1, autoscale = FALSE),
  prior = normal(0, 10, autoscale = FALSE),
  chains = 2, iter = 3000, warmup = 1000, seed = 123
)
```

```
SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.00012 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.2 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.285 seconds (Warm-up)
Chain 1:                3.033 seconds (Sampling)
Chain 1:                3.318 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 3.1e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.31 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
```

```
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.288 seconds (Warm-up)
Chain 2:                3.167 seconds (Sampling)
Chain 2:                3.455 seconds (Total)
Chain 2:
```

```r
delta_noninf <- as.data.frame(fit_noninf)$`new_algoB`

# Fit informative model
fit_inf <- stan_glm(
  minutes ~ new_algo,
  data = df_combined,
  family = gaussian(),
  prior_intercept = normal(4, 1, autoscale = FALSE),
  prior = normal(20, 10, autoscale = FALSE),
  chains = 2, iter = 3000, warmup = 1000, seed = 123
)
```

```
SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 3.9e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.39 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.553 seconds (Warm-up)
```

```
Chain 1:                3.301 seconds (Sampling)
Chain 1:                3.854 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 3.1e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.31 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.318 seconds (Warm-up)
Chain 2:                3.119 seconds (Sampling)
Chain 2:                3.437 seconds (Total)
Chain 2:
```

```r
delta_inf <- as.data.frame(fit_inf)$`new_algoB`

# Define threshold for 5% improvement
mean_minutes_A <- mean(y_A)
threshold_5pc <- 0.05 * mean_minutes_A

# ------------------------
# PRINT RESULTS
# ------------------------
cat("--- Non-informative Prior ---\n")
```

```
--- Non-informative Prior ---
```

```r
print(c(
  mean = mean(delta_noninf),
  sd = sd(delta_noninf),
  hdi_2.5 = quantile(delta_noninf, 0.025),
  hdi_97.5 = quantile(delta_noninf, 0.975)
))
```

```
          mean             sd   hdi_2.5.2.5% hdi_97.5.97.5%
     10.483369       6.727302      -2.657194      23.311492
```

```r
cat(sprintf("P(Delta > 0)         = %.4f\n", mean(delta_noninf > 0)))
```

```
P(Delta > 0)         = 0.9390
```

```r
cat(sprintf("P(Delta > 0.1)       = %.4f\n", mean(delta_noninf > 0.1)))
```

```
P(Delta > 0.1)       = 0.9377
```

```r
cat(sprintf("P(Delta > 5%% of A)   = %.4f\n\n", mean(delta_noninf > threshold_5pc)))
```

```
P(Delta > 5% of A)   = 0.0742
```

```r
cat("--- Informative Prior ---\n")
```

```
--- Informative Prior ---
```

```r
print(c(
  mean = mean(delta_inf),
  sd = sd(delta_inf),
  hdi_2.5 = quantile(delta_inf, 0.025),
  hdi_97.5 = quantile(delta_inf, 0.975)
))
```

```
          mean             sd   hdi_2.5.2.5% hdi_97.5.97.5%
     19.435374       6.696254       6.044236      32.635245
```

```r
cat(sprintf("P(Delta > 0)          = %.4f\n", mean(delta_inf > 0)))
```

```
P(Delta > 0)          = 0.9978
```

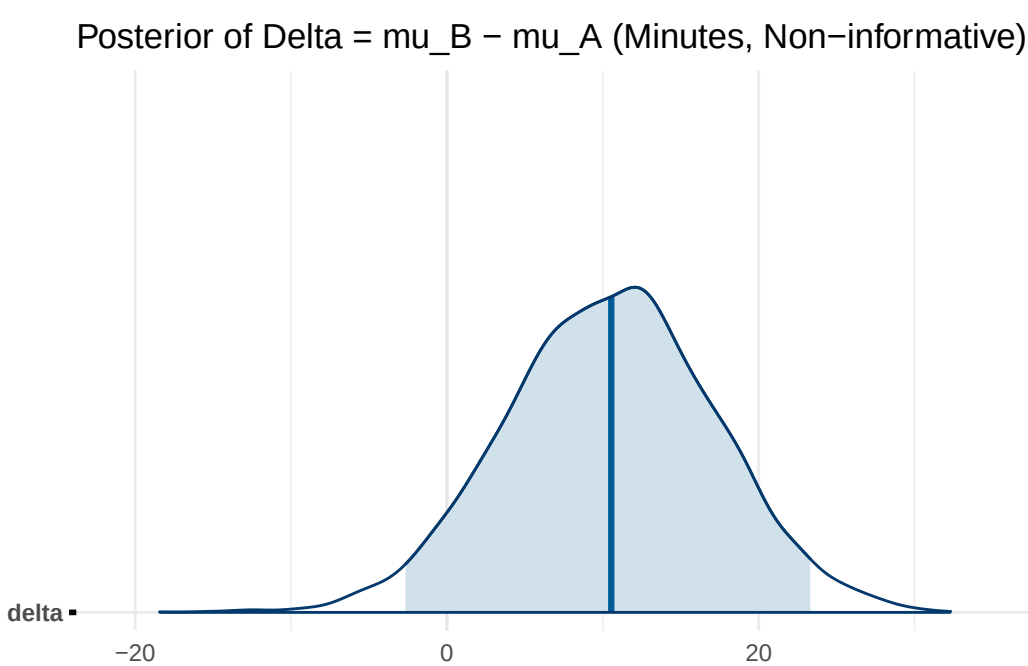
```r
cat(sprintf("P(Delta > 0.1)        = %.4f\n", mean(delta_inf > 0.1)))
```

```
P(Delta > 0.1)        = 0.9978
```

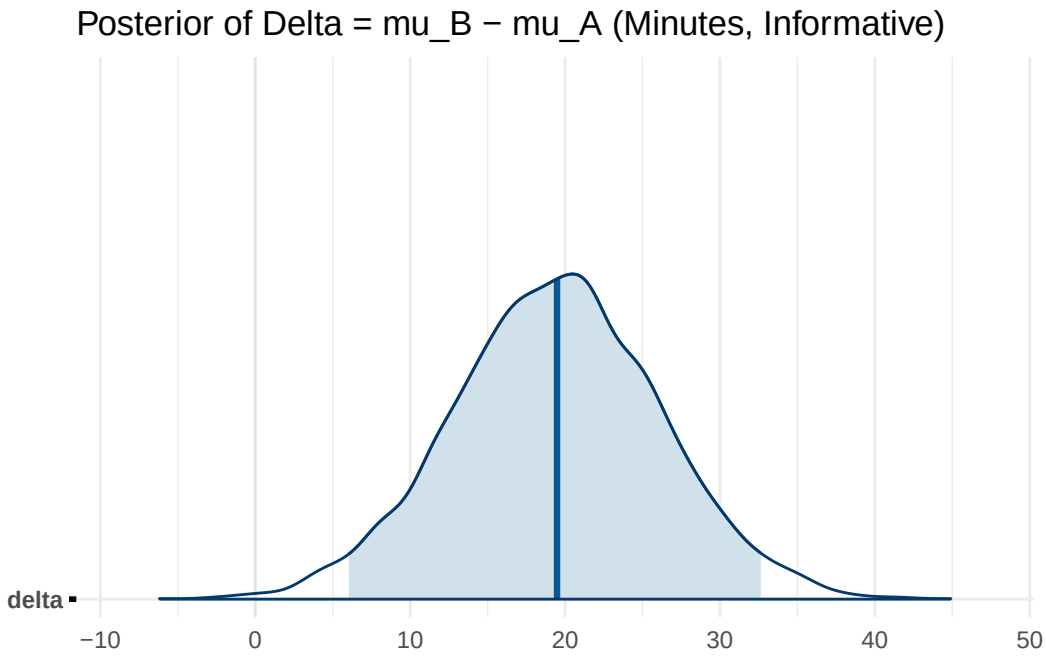
```r
cat(sprintf("P(Delta > 5%% of A)    = %.4f\n", mean(delta_inf > threshold_5pc)))
```

```
P(Delta > 5% of A)    = 0.4700
```

```r
# -----------------------
# PLOTS
# -----------------------
p1 <- mcmc_areas(as.matrix(data.frame(delta = delta_noninf)), prob = 0.95) +
  ggtitle("Posterior of Delta = mu_B - mu_A (Minutes, Non-informative)")
p2 <- mcmc_areas(as.matrix(data.frame(delta = delta_inf)), prob = 0.95) +
  ggtitle("Posterior of Delta = mu_B - mu_A (Minutes, Informative)")

print(p1)
```



```r
print(p2)
```



Interpretation for non-informative:

The Bayesian model estimated an average effect of 10.48minutes and 94% times its saying that the value is greater than 0 or positive and same for P(Delta > 0.1) as well. The HDI says that it has 95% probability that the true effect lies between the interval(-2.65, 23.31).

- There is a **7.4% probability** that users listen **5min more** as P(Delta > 5% of A) = 0.0742

Interpretation for informative:

In our problem, we have set a boundaries, 20min improvement with a SD of 10minutes. After using an informative prior, the model estimated average effect is 19.44 minutes, with a 99.8% probability that the effect is positive. The 95% HDI(6.04, 32.64), indicates high probability that the effect is both positive. In terms of our business problem the value is extremely significant.

- There is a **47% probability** that users listen **5min more** as P(Delta > 5% of A) = 0.4700.

Churn (Non-informative, Informative)

We use a Bayesian logistic regression via "stan_glm()" with a `binomial(link = "logit")` likelihood to model the binary churn outcome.

```
library(rstanarm)
library(bayesplot)
library(ggplot2)

theme_set(theme_minimal(base_family = "sans"))  # PDF-safe font

# Load and prepare churn data
df <- read.csv("data_new_algo.csv")
y_A <- df[df$new_algo == 0, "churn"]
y_B <- df[df$new_algo == 1, "churn"]

df_churn <- data.frame(
  churn = c(y_A, y_B),
  new_algo = c(rep("A", length(y_A)), rep("B", length(y_B)))
)
df_churn$new_algo <- relevel(as.factor(df_churn$new_algo), ref = "A")

# -------------------------------
# 1. Non-informative model
# -------------------------------
set.seed(123)
fit_churn_noninf <- stan_glm(
  churn ~ new_algo,
  data = df_churn,
  family = binomial(link = "logit"),
  prior_intercept = student_t(7, 0, 10, autoscale = FALSE),
  prior = student_t(7, 0, 10, autoscale = FALSE),
  chains = 2, iter = 3000, warmup = 1000, seed = 123
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.000902 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 9.02 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
```

```
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 7.093 seconds (Warm-up)
Chain 1:                11.883 seconds (Sampling)
Chain 1:                18.976 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 0.000774 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 7.74 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 5.591 seconds (Warm-up)
Chain 2:                11.407 seconds (Sampling)
Chain 2:                16.998 seconds (Total)
Chain 2:
```

```r
post_noninf <- as.data.frame(fit_churn_noninf)
p_A_noninf <- plogis(post_noninf$`(Intercept)`)
p_B_noninf <- plogis(post_noninf$`(Intercept)` + post_noninf$`new_algoB`)
delta_p_noninf <- p_B_noninf - p_A_noninf

# -------------------------------
# 2. Informative model
# -------------------------------
fit_churn_inf <- stan_glm(
  churn ~ new_algo,
  data = df_churn,
  family = binomial(link = "logit"),
  prior_intercept = normal(-2.2, 0.5, autoscale = FALSE),
  prior = normal(-0.2, 0.1, autoscale = FALSE),
  chains = 2, iter = 3000, warmup = 1000, seed = 123
)
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 0.001096 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 10.96 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 1: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 1: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 1: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 1: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 1: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 1: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 1: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 1: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 1: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 1: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 1: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 6.991 seconds (Warm-up)
Chain 1:                11.45 seconds (Sampling)
Chain 1:                18.441 seconds (Total)
Chain 1:
```

```
SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 0.000801 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 8.01 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 3000 [  0%]  (Warmup)
Chain 2: Iteration:  300 / 3000 [ 10%]  (Warmup)
Chain 2: Iteration:  600 / 3000 [ 20%]  (Warmup)
Chain 2: Iteration:  900 / 3000 [ 30%]  (Warmup)
Chain 2: Iteration: 1001 / 3000 [ 33%]  (Sampling)
Chain 2: Iteration: 1300 / 3000 [ 43%]  (Sampling)
Chain 2: Iteration: 1600 / 3000 [ 53%]  (Sampling)
Chain 2: Iteration: 1900 / 3000 [ 63%]  (Sampling)
Chain 2: Iteration: 2200 / 3000 [ 73%]  (Sampling)
Chain 2: Iteration: 2500 / 3000 [ 83%]  (Sampling)
Chain 2: Iteration: 2800 / 3000 [ 93%]  (Sampling)
Chain 2: Iteration: 3000 / 3000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 6.708 seconds (Warm-up)
Chain 2:                12.856 seconds (Sampling)
Chain 2:                19.564 seconds (Total)
Chain 2:
```

```r
post_inf <- as.data.frame(fit_churn_inf)
p_A_inf <- plogis(post_inf$`(Intercept)`)
p_B_inf <- plogis(post_inf$`(Intercept)` + post_inf$`new_algoB`)
delta_p_inf <- p_B_inf - p_A_inf

# -------------------------------
# Print summaries
# -------------------------------
cat("--- Churn (Non-informative Prior) ---\n")
```

```
--- Churn (Non-informative Prior) ---
```

```r
print(c(
  mean = mean(delta_p_noninf),
  sd = sd(delta_p_noninf),
  hdi_2.5 = quantile(delta_p_noninf, 0.025),
```

```r
  hdi_97.5 = quantile(delta_p_noninf, 0.975)
))
```

```
          mean             sd   hdi_2.5.2.5% hdi_97.5.97.5%
  -0.024988442    0.005639854   -0.036061781   -0.014214945
```

```r
cat(sprintf("P(Delta_p > 0)       = %.4f\n", mean(delta_p_noninf > 0)))
```

```
P(Delta_p > 0)       = 0.0000
```

```r
cat(sprintf("P(Delta_p < 0)       = %.4f\n", mean(delta_p_noninf < 0)))
```

```
P(Delta_p < 0)       = 1.0000
```

```r
cat(sprintf("P(Delta_p < -0.02)   = %.4f\n\n", mean(delta_p_noninf < -0.02)))
```

```
P(Delta_p < -0.02)   = 0.8120
```

```r
cat("--- Churn (Informative Prior) ---\n")
```

```
--- Churn (Informative Prior) ---
```

```r
print(c(
  mean = mean(delta_p_inf),
  sd = sd(delta_p_inf),
  hdi_2.5 = quantile(delta_p_inf, 0.025),
  hdi_97.5 = quantile(delta_p_inf, 0.975)
))
```

```
          mean             sd   hdi_2.5.2.5% hdi_97.5.97.5%
  -0.022007629    0.004719803   -0.031096429   -0.012935473
```

```r
cat(sprintf("P(Delta_p > 0)       = %.4f\n", mean(delta_p_inf > 0)))
```

```
P(Delta_p > 0)       = 0.0000
```

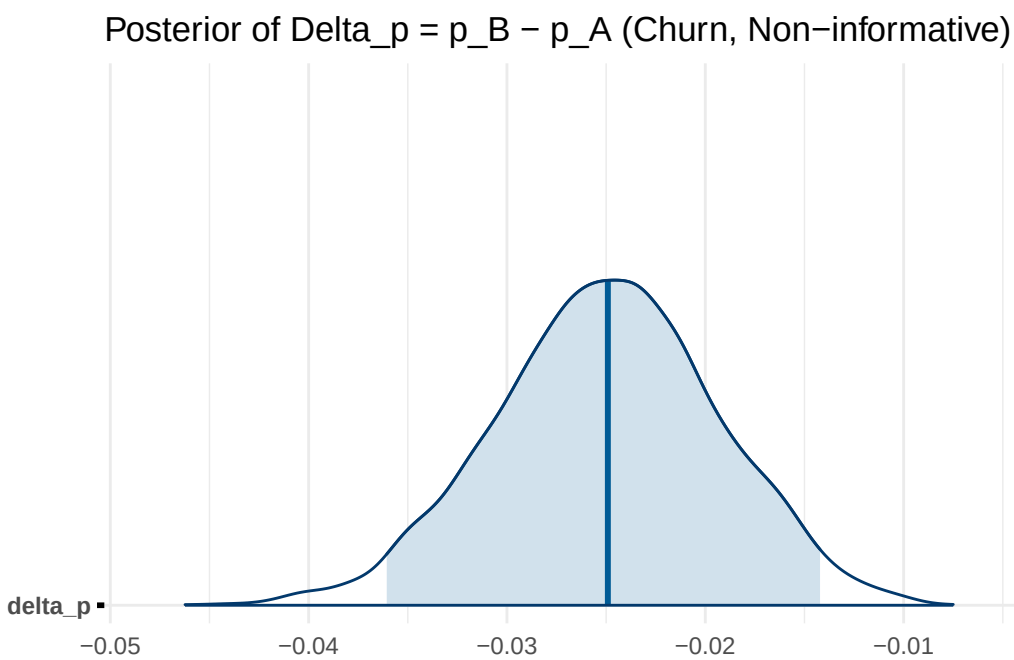
```r
cat(sprintf("P(Delta_p < 0)       = %.4f\n", mean(delta_p_inf < 0)))
```

```
P(Delta_p < 0)       = 1.0000
```

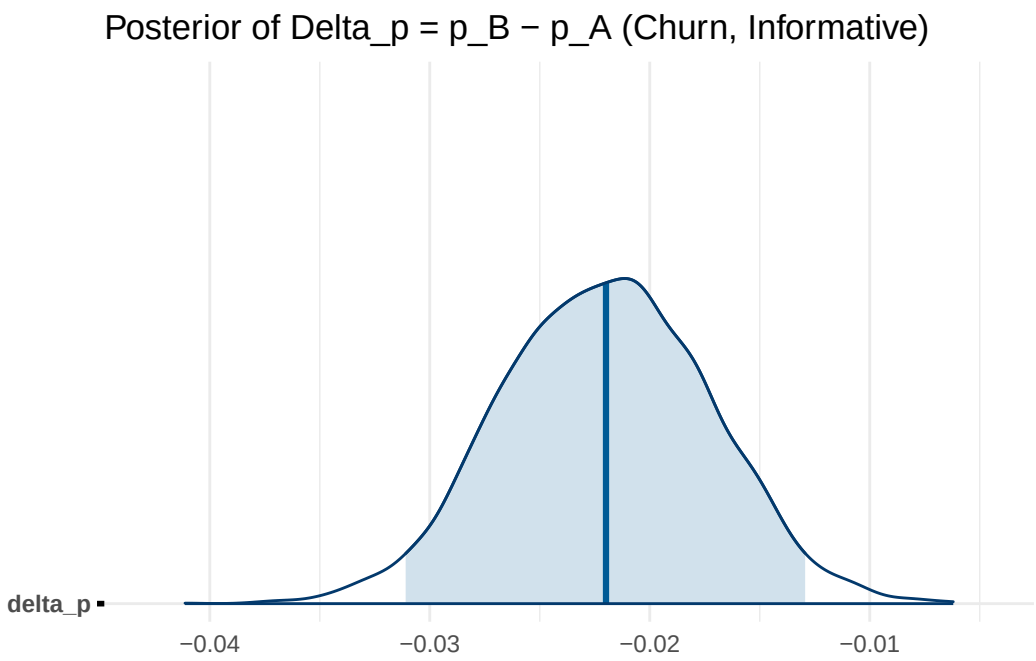
```r
cat(sprintf("P(Delta_p < -0.02)   = %.4f\n", mean(delta_p_inf < -0.02)))
```

```
P(Delta_p < -0.02)   = 0.6637
```

```r
# -------------------------------
# Plot posterior distributions
# -------------------------------
p1 <- mcmc_areas(as.matrix(data.frame(delta_p = delta_p_noninf)), prob = 0.95) +
  ggtitle("Posterior of Delta_p = p_B - p_A (Churn, Non-informative)")
p2 <- mcmc_areas(as.matrix(data.frame(delta_p = delta_p_inf)), prob = 0.95) +
  ggtitle("Posterior of Delta_p = p_B - p_A (Churn, Informative)")

print(p1)
```




```r
print(p2)
```



Interpretation for non-informative:

The model estimates an average reduction in churn of -0.025, with a 100% posterior probability that the effect is negative. The 95% HDI (-0.036,-0.014) indicates strong evidence that the new algorithm reduces churn.

- There is a **81% probability** that churn **2%** as P(Delta_p < -0.01) = 0.8120

Interpretation for informative:

Using an informative prior, the model estimates a churn reduction of -0.022(slight less), with a 100% probability the effect is negative. The 95% HDI (-0.031, -0.013) confirms high confidence that users are less churning.

- There is a **66% probability** that churn **2%** as P(Delta_p < -0.01) = 0.6637

References